UNITÀ 2 · PROGETTARE E UTILIZZARE WEB SERVICE · LEZIONE 5

I web service e il protocollo HTTP

Ogni web service, SOAP o REST, viaggia dentro HTTP. Oggi apriamo la busta e leggiamo riga per riga che cosa si dicono client e server.

5G TPSI Richieste, risposte, metodi, codici di stato, header, MIME

PER COMINCIARE

Quale di questi codici di errore hai visto con i tuoi occhi?

SONDAGGIO

- **A** 404
- **B** 403
- **C** 500
- **D** 503
- **E** 429

MENTI CHE CI HANNO PRECEDUTO

Dal CERN al tuo telefono

> **Il potere del Web sta nella sua universalità.**
>
> **Tim Berners-Lee**, inventore del Web e direttore del W3C, al lancio della Web Accessibility Initiative, 1997

1991

HTTP/0.9: una sola riga, `GET /pagina`, e torna l'HTML.

1996-97

HTTP/1.0 e 1.1: header, codici di stato, metodi, `Host`.

2015

HTTP/2: binario, più richieste insieme sulla stessa connessione.

2022

HTTP/3: viaggia su QUIC, sopra UDP invece che TCP.

ANATOMIA

Una richiesta e la sua risposta

```
POST /api/voti HTTP/1.1
Host: registro.example.org
Content-Type: application/json
Content-Length: 40
Authorization: Bearer eyJhbGci…

{"studente":7,"materia":"TPSI","voto":9}
```

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/voti/42
Content-Length: 48

{"id":42,"studente":7,"materia":"TPSI",
 "voto":9}
```

1 · Prima riga

Richiesta: metodo, percorso, versione. Risposta: versione, codice, frase.

2 · Header

Righe `Nome: valore`, poi una **riga vuota**.

3 · Corpo

Facoltativo. Content-Length dice quanti byte.

I METODI

Che cosa chiede il client

Metodo	Uso tipico	Corpo	Sicuro	Idempotente
`GET`	Leggere una risorsa	No	Sì	Sì
`HEAD`	Come GET, ma solo gli header	No	Sì	Sì
`POST`	Creare, inviare un modulo, eseguire un'azione	Sì	No	No
`PUT`	Sostituire per intero una risorsa	Sì	No	Sì
`PATCH`	Modificare una parte	Sì	No	Non garantito
`DELETE`	Cancellare	Di solito no	No	Sì
`OPTIONS`	Chiedere che cosa è permesso	No	Sì	Sì

Sicuro: non modifica lo stato del server. **Idempotente**: ripetuto più volte, lascia il server come una volta sola. È quello che serve per **riprovare** (lezione 2).

QUIZ A TRABOCCHETTO

`DELETE /api/voti/42` inviato due volte: la prima risponde 204, la seconda 404. È idempotente?

QUIZ

- **A** No: le due risposte sono diverse
- **B** No: DELETE non è mai idempotente
- **C** Sì: conta l'effetto sul server, non la risposta
- **D** Solo se la connessione è HTTPS

I CODICI DI STATO

Cinque famiglie, una regola: la prima cifra

2xx

Successo

200 OK · **201** Created · **204** No Content

3xx

Vai altrove

301 spostata per sempre · **302** temporaneamente · **304** Not Modified, usa la cache

4xx

Errore del client

400 richiesta malformata · **401** chi sei? · **403** non puoi · **404** non c'è · **405** metodo no · **429** troppe

5xx

Errore del server

500 errore interno · **502** il gateway non riceve risposta valida · **503** non disponibile

La famiglia 1xx (informazioni, per esempio **101** Switching Protocols per i WebSocket) si vede raramente.

PUZZLE

Che codice risponde il server?

201	Il server è in manutenzione programmata
304	Hai mandato un JSON con una parentesi in meno
400	La pagina non è cambiata dalla tua ultima visita
405	Hai fatto 100 richieste in un secondo
429	Hai fatto DELETE su un percorso che accetta solo GET
503	Hai inviato un nuovo voto ed è stato salvato

METTITI ALLA PROVA

Sei entrato nel registro come studente e apri la pagina riservata ai docenti. Che codice è corretto?

QUIZ

- **A** 401 Unauthorized
- **B** 403 Forbidden
- **C** 400 Bad Request
- **D** 500 Internal Server Error

GLI HEADER

Le informazioni che accompagnano il messaggio

Nella richiesta	A che cosa serve
`Host`	Quale sito, sullo stesso server
`Accept`	Che formati accetto
`Authorization`	Chi sono: token, credenziali
`User-Agent`	Quale programma chiede
`Cookie`	Dati ricordati per questo sito

Nella risposta	A che cosa serve
`Content-Type`	Che cosa c'è nel corpo
`Location`	Dove andare o dov'è il nuovo
`Cache-Control`	Per quanto tenerlo in cache
`Set-Cookie`	Ricordati questo dato
`Retry-After`	Quando riprovare

> HTTP è **senza stato** (stateless): ogni richiesta è indipendente. Se il server deve ricordarsi di te, la "memoria" viaggia in ogni richiesta: un **cookie** di sessione o un **token** nell'header Authorization.

I TIPI MIME

`tipo/sottotipo`: che cosa c'è nel corpo

Tipo MIME	Contenuto
`text/html; charset=utf-8`	Pagina web
`application/json`	Dati JSON di un'API
`application/xml, text/xml`	Dati XML, SOAP 1.1
`application/soap+xml`	Messaggi SOAP 1.2
`image/png, image/jpeg`	Immagini
`application/x-www-form-urlencoded`	Modulo HTML inviato
`multipart/form-data`	Modulo con file allegati

Il nome viene dalla posta elettronica: **MIME** = Multipurpose Internet Mail Extensions.

Negoziazione

Il client chiede con `Accept: application/json`, il server sceglie e dichiara con `Content-Type`. Se non sa accontentarlo: **406**.

Se il client manda un tipo che il server non sa leggere: **415** Unsupported Media Type.

AL COMPUTER · 20 minuti

HTTP dal vivo con i DevTools

1. Apri **it.wikipedia.org**, premi F12, scheda **Network**, ricarica.
2. Clicca la prima richiesta (il documento). In **Headers** annota metodo, codice di stato, versione del protocollo e Content-Type.
3. Trova un file CSS, uno JavaScript e un'immagine: che Content-Type hanno?
4. Ricarica **senza** svuotare la cache: cerca un **304** o la dicitura "memory cache"/"disk cache".
5. Nella barra degli indirizzi scrivi `http://it.wikipedia.org` (senza s): che codice vedi come prima richiesta?
6. Annota tutto in **http-devtools-cognome.txt**.

Controllo: documento **GET**, **200**, `text/html; charset=utf-8`; CSS `text/css`, JS `text/javascript` o `application/javascript`. Con http:// trovi un redirect **3xx** con Location verso https://. Il protocollo è spesso h2 o h3.

AL COMPUTER · 20 minuti

HTTP riga per riga con curl

```
curl -v http://www.wikipedia.org -o NUL
curl -I https://www.wikipedia.org
curl -i https://it.wikipedia.org/wiki/Pagina_che_non_esiste_5G -o NUL -w "%{http_code}\n"
curl -I -H "Accept-Language: en" https://www.wikipedia.org
```

1. Esegui i comandi nel Prompt dei comandi (PowerShell: `curl.exe`). Con `-v` le righe con **>** sono la richiesta, quelle con **<** la risposta.
2. Per ogni comando annota la prima riga della risposta e due header che ti sembrano utili.
3. Spiega in una frase che cosa fanno `-v`, `-I`, `-i`, `-H`.
4. Salva in **http-curl-cognome.txt**.

Controllo: la richiesta contiene `GET / HTTP/1.1`, `Host`, `User-Agent: curl/…`, `Accept: */*`. Il primo comando riceve un redirect **3xx** con Location https; il secondo **200**; il terzo **404**.

AL COMPUTER · 25 minuti

Il tuo server risponde con codici e header giusti

```
var srv = HttpServer.create(new InetSocketAddress(8081), 0);
srv.createContext("/orario", ex -> {
  System.out.println(ex.getRequestMethod() + " " + ex.getRequestURI()
      + "  Accept: " + ex.getRequestHeaders().getFirst("Accept"));
  if (!ex.getRequestMethod().equals("GET")) {
    ex.getResponseHeaders().set("Allow", "GET");
    ex.sendResponseHeaders(405, -1);        // -1 = nessun corpo
  } else {
    byte[] b = "{\"classe\":\"5G\",\"prima_ora\":\"TPSI\"}".getBytes();
    ex.getResponseHeaders().set("Content-Type", "application/json");
    ex.sendResponseHeaders(200, b.length);
    ex.getResponseBody().write(b);
  }
  ex.close();
});
srv.createContext("/vecchio", ex -> {
  ex.getResponseHeaders().set("Location", "/orario");
  ex.sendResponseHeaders(301, -1);
  ex.close();
});
srv.start();
```

1. **ServerHttp.java**: codice nel `main`, import di `com.sun.net.httpserver.*` e `java.net.InetSocketAddress`.
2. Prova con `curl -i` su `/orario`, poi `-X DELETE`, `-I`, `/vecchio` con e senza `-L`, e `/nulla`.

Controllo: GET **200** con il JSON; DELETE **405** e `Allow: GET`; /vecchio **301** con Location (con –L arriva il JSON); /nulla **404**. E `-I`?

OGGI

HTTP sotto le app di intelligenza artificiale





SFIDA FINALE

Il lucchetto di HTTP



- 1ª cifra: la prima cifra dei codici di redirect
- 2ª cifra: quanti tra GET, POST, PUT e DELETE sono idempotenti
- 3ª cifra: l'ultima cifra del codice "Not Found"
- 4ª cifra: l'ultima cifra del codice "Too Many Requests"

```
codice
```

BIGLIETTO D'USCITA

Completa prima di uscire

Tra gli header e il corpo di un messaggio HTTP c'è una riga _____.

Il codice per una risorsa appena creata è _____.

Credenziali mancanti o non valide: _____. Autenticato ma senza permesso: _____.

L'header che dice il formato del corpo si chiama _____.

HTTP non ricorda le richieste precedenti: è senza _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- Messaggio: prima riga, header, riga vuota, corpo
- GET, PUT, DELETE idempotenti; POST no
- 2xx ok, 3xx altrove, 4xx colpa del client, 5xx del server
- 401 non autenticato, 403 non autorizzato
- Content-Type e Accept: tipi MIME e negoziazione
- Stateless: cookie e token portano la memoria

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.